qual_by_position.py 사용법 설명서

> lab11 · 생물정보학실험
> 대상 파일: `lab11/src/qual_by_position.py`

---

1. 프로그램의 목적과 기능

`qual_by_position.py`는 **FASTQ 파일에서 read의 각 염기 위치(1번째, 2번째, 3번째 …)마다 평균 품질 점수를 계산하는 프로그램**입니다.

시퀀싱 기계는 DNA를 앞에서부터 한 염기씩 읽어 나가는데, 보통 **뒤로 갈수록 정확도가 떨어집니다.** 이 프로그램을 쓰면 "몇 번째 위치부터 품질이 나빠지는지"를 숫자로 확인할 수 있습니다.

비교	`qual_summary.py`	`qual_by_position.py`
평균을 내는 방향	read 하나 단위 (가로)	위치 하나 단위 (세로)
결과	숫자 1개	위치마다 1줄 (표)
알 수 있는 것	데이터 전체가 좋은가?	**어느 위치**가 나쁜가?

프로그램이 하는 일은 세 단계입니다.

1. FASTQ 파일을 읽어서 (이름, 서열, 품질) 묶음의 목록을 만든다. → `read_fastq()`
2. 모든 read를 돌면서 위치별로 **품질 점수 합계**와 **read 개수**를 센다.
3. 위치별로 `합계 ÷ 개수`를 계산해 표로 출력한다.

---

2. 실행 명령어와 사용 방법

`lab11` 폴더에서 아래처럼 실행합니다.

```
cd lab11
python3 src/qual_by_position.py 〈FASTQ 파일 경로〉
```

예시:

```
python3 src/qual_by_position.py data/SRR1553607_1.fastq
python3 src/qual_by_position.py data/trimmed.fastq
```

결과가 100줄이 넘으므로 앞부분만 보거나, 파일로 저장하면 편합니다.

```
python3 src/qual_by_position.py data/SRR1553607_1.fastq ¦ head -20
python3 src/qual_by_position.py data/SRR1553607_1.fastq > doc/qual_pos_1.tsv
```

- 입력 파일은 **하나만** 받습니다. (`sys.argv[1]`)
- 출력은 **탭(Tab)으로 구분된 표**라서 `.tsv`로 저장하면 엑셀에서 바로 열 수 있습니다.

> **주의**
>
> - 파일 이름을 빼먹으면 `IndexError`가 납니다.
> - FASTQ는 **4줄이 한 read**라는 규칙을 그대로 믿고 읽습니다. 파일 중간에 빈 줄이 있거나 줄 수가 4의 배수가 아니면 오류가 나거나 결과가 틀어집니다.
> - 파일 전체를 한꺼번에 메모리에 올리므로(`readlines()`), 아주 큰 파일에는 적합하지 않습니다. (실습 파일은 약 7초면 끝납니다.)

---

3. FASTQ 파일에서 품질 점수를 읽는 과정

FASTQ의 구조

FASTQ 파일은 **read 하나가 4줄**로 이루어져 있습니다. 실습 파일의 첫 read를 보면:

```
@SRR1553607.1 1 length=101          ← 1줄: @ + read 이름
GTTAGCGTTGTTGATCGCGACGCAACAAC...    ← 2줄: 염기 서열
+SRR1553607.1 1 length=101          ← 3줄: + (구분선)
BB@FFFFFHHHHHJJJJJJJJJJJJJJJJ...    ← 4줄: 품질 문자열
```

4번째 줄의 문자 하나하나가 2번째 줄 염기 하나하나의 품질입니다. 그래서 **서열과 품질 문자열의 길이는 항상 같습니다.**

`read_fastq()` 함수

```
def read_fastq(filename):
    records = []
    with open(filename) as f:
        lines = f.readlines()          # 파일 전체를 줄 단위 리스트로

    for i in range(0, len(lines), 4):  # 0, 4, 8, ... 4줄씩 건너뛰며
        name = lines[i].rstrip()[1:]   # 1줄: 맨 앞 '@' 제거
        seq = lines[i + 1].rstrip()    # 2줄: 서열
        qual = lines[i + 3].rstrip()   # 4줄: 품질 문자열
        records.append((name, seq, qual))

    return records
```

- `rstrip()` : 줄 끝의 줄바꿈 문자(`\n`)를 지웁니다.
- `[1:]` : 이름 맨 앞의 `@`를 빼고 저장합니다.
- 3번째 줄(`+`)은 쓸 일이 없어서 건너뜁니다.
- 결과는 `[(이름, 서열, 품질), (이름, 서열, 품질), ...]` 형태의 리스트입니다.

---

4. `ord(char) - 33`의 의미

품질 점수는 원래 **숫자**(예: 30, 40)지만, FASTQ는 한 염기에 한 글자만 쓰기 위해 **숫자를 문자로 바꿔서** 저장합니다. 이 방식을 **Phred+33**이라고 합니다.

- `ord(char)` : 문자를 컴퓨터 안의 번호(**ASCII 코드**)로 바꿉니다.
- `- 33` : 저장할 때 33을 더해 두었으므로, 다시 33을 빼서 원래 점수로 돌립니다.

문자	`ord()` 값	`- 33` = 품질 점수	틀릴 확률
`!`	33	0	1 (전혀 못 믿음)
`+`	43	10	1/10
`5`	53	20	1/100
`?`	63	30	1/1,000
`B`	66	33	약 1/2,000
`J`	74	41	약 1/12,600

왜 33인가? ASCII 0~32번은 공백·줄바꿈 같은 "눈에 안 보이는 문자"라서 파일에 쓰기 곤란합니다. 그래서 눈에 보이는 첫 문자인 `!`(33번)를 점수 0으로 정했습니다.

품질 점수 Q와 오류 확률 P의 관계는 `Q = -10 × log10(P)` 입니다. 쉽게 말해 **점수가 10 오르면 틀릴 확률이 10분의 1로 줄어듭니다.** 보통 **Q30 이상이면 좋은 품질, Q20 미만이면 나쁜 품질**로 봅니다.

---

5. 각 염기 위치별 평균 품질 점수를 계산하는 방법

핵심 아이디어는 **위치마다 "점수 합계"와 "read 수"를 따로 모아 두었다가, 마지막에 나누는 것**입니다.

```
quality_sums = []     # quality_sums[i]   = i번 위치 점수의 합
quality_counts = []   # quality_counts[i] = i번 위치를 가진 read 수

for name, seq, qual in reads:            # read 하나씩
    for i, char in enumerate(qual):      # 품질 문자 하나씩 (i = 위치 번호)
        if i >= len(quality_sums):       # 처음 보는 위치면 칸 추가
            quality_sums.append(0)
            quality_counts.append(0)

        score = ord(char) - 33           # 문자 → 점수
        quality_sums[i] += score         # 합계에 더하고
        quality_counts[i] += 1           # 개수 1 증가
```

- `enumerate(qual)` 은 문자와 함께 **위치 번호 i(0부터 시작)**를 같이 줍니다.

작은 예시로 따라가 보기

read 3개, 품질 문자열이 아래와 같다고 해 봅시다.

```
read A:  I I 5      → 점수 40 40 20
read B:  I ? 5      → 점수 40 30 20
read C:  ? ?        → 점수 30 30      (길이 2)
```

위치	합계 (quality_sums)	개수 (quality_counts)	평균
1	40+40+30 = 110	3	36.67
2	40+30+30 = 100	3	33.33
3	20+20 = 40	2	20.00

출력 부분

```
print("위치\t평균 품질 점수\tRead 수")
for i in range(len(quality_sums)):
    mean = quality_sums[i] / quality_counts[i]
    print(f"{i + 1}\t{mean:.2f}\t{quality_counts[i]}")
```

- 파이썬은 0부터 세지만 사람은 1부터 세므로 `i + 1`로 출력합니다.
- `:.2f`는 소수점 둘째 자리까지 보여 줍니다.

---

6. read 길이가 서로 다를 때 처리하는 방법

트리밍(trimming)을 하면 read마다 길이가 달라집니다. 이 프로그램은 두 가지 장치로 이 문제를 해결합니다.

1. **리스트를 미리 크기를 정하지 않고, 필요할 때 늘립니다.** `if i >= len(quality_sums)` 조건 덕분에, 지금까지 본 것보다 긴 read가 나오면 그때 칸을 추가합니다. 그래서 가장 긴 read의 길이만큼 결과 줄이 생깁니다.

2. **위치마다 read 수를 따로 셉니다.** 짧은 read는 자기 길이까지만 `quality_counts`를 올립니다. 그래서 위치별 평균은 **"그 위치까지 실제로 있는 read들만의 평균"**이 됩니다.

만약 모든 위치를 전체 read 수로 나눴다면, 짧은 read가 없는 뒷부분은 0점이 섞인 것처럼 계산되어 평균이 실제보다 낮게 나왔을 것입니다.

> **해석할 때 주의:** 뒤쪽 위치일수록 read 수가 줄어듭니다. read 수가 아주 적은 위치의 평균은 몇 개 read에 크게 흔들릴 수 있으니, 출력의 세 번째 열(`Read 수`)도 함께 확인하세요.

---

7. 실제 실행 결과와 해석

(1) 원본 데이터: `SRR1553607_1.fastq`

```
python3 src/qual_by_position.py data/SRR1553607_1.fastq
```

```
위치  평균 품질 점수    Read 수
1   32.04   203445
2   32.17   203445
3   32.28   203445
4   35.85   203445
...
9   37.27   203445
...
14  38.47   203445    ← 가장 높음
...
98  29.13   203445
99  29.03   203445
100 28.93   203445
101 28.25   203445    ← 가장 낮음
```

- 총 **203,445개 read**, 모두 길이 **101**이라 모든 위치의 read 수가 같습니다.
- **처음 1~3번 위치**는 32점대로 약간 낮습니다. 시퀀싱 시작 직후 기계가 안정되기 전이라 흔히 보이는 현상입니다.
- **10~20번 위치 부근**이 38점 안팎으로 가장 좋습니다.
- **뒤로 갈수록 점점 떨어져** 마지막 101번 위치는 28.25점입니다. Illumina 시퀀싱의 전형적인 모습입니다.

- 그래도 마지막까지 평균이 Q28 이상이므로, 전체적으로는 **양호한 데이터**입니다.

(2) 트리밍 후 데이터: `trimmed.fastq`

```
python3 src/qual_by_position.py data/trimmed.fastq
```

```
위치  평균 품질 점수    Read 수
1   32.28   188357
...
50  34.92   188357
51  34.74   188343    ← 여기서부터 read 수가 줄어듦
52  34.58   188304
53  34.41   188281
...
100 30.92   171032
101 30.17   171032
```

- fastp로 품질이 낮은 read와 끝부분을 잘라냈기 때문에 read 수가 **188,357개**로 줄었고, 길이가 **50~101**로 다양해졌습니다.
- 1~50번 위치는 모든 read가 가지고 있어 read 수가 같지만, **51번부터는 짧은 read가 빠지면서 read 수가 줄어듭니다.** (6장의 처리 방식이 실제로 동작하는 모습)
- 마지막 위치 평균이 **28.25 → 30.17**로 올라갔습니다. 품질 나쁜 끝부분이 잘려 나갔기 때문입니다. 즉 트리밍이 효과가 있었다는 뜻입니다.

---

8. FastQC의 Per base sequence quality 그래프와의 관계

FastQC 보고서(`doc/SRR1553607_1_fastqc.html`)의 **Per base sequence quality** 그래프는 이 프로그램과 **같은 일**을 그림으로 보여 줍니다.

FastQC 그래프	`qual_by_position.py`
가로축 (Position in read)	1열 `위치`
파란 선 (Mean)	2열 `평균 품질 점수`
노란 상자 (사분위수), 빨간 선 (중앙값)	계산하지 않음
배경색 (초록 ≥28, 주황 20~28, 빨강 <20)	없음 → 숫자를 보고 직접 판단

실제로 FastQC 결과 파일(`fastqc_data.txt`)의 평균값과 비교해 보면 **값이 같습니다.**

위치	FastQC Mean	이 프로그램
1	32.0407	32.04
4	35.8499	35.85
9	37.2712	37.27

차이점도 하나 있습니다. FastQC는 뒤쪽 위치를 `10-11`, `12-13`처럼 **두세 개씩 묶어서** 보여 줍니다. 예를 들어 trimmed 데이터에서 FastQC의 `100-101` 값은 30.55인데, 이는 이 프로그램의 100번(30.92)과 101번(30.17) 값의 평균과 같습니다. 이 프로그램은 묶지 않고 **한 위치씩 모두** 보여 줍니다.

정리: `qual_by_position.py`는 FastQC 그래프의 **파란 평균선을 직접 계산해 본 프로그램**입니다. 그래프로는 전체 모양을 한눈에 보고, 이 프로그램으로는 정확한 숫자를 확인하면 됩니다.